

From Linear Regression to Neural Networks

Neurons, Linear Layers, Convolutions, Normalization and Attention

Nicolas Caron

FEMTO-ST

May 2026

Course Outline

- 1 Recall: Gradient Descent
- 2 Training Vocabulary
- 3 Linear Regression as a Neuron
- 4 Activation Functions
- 5 Convolutional layer
- 6 Normalization Layers
- 7 Attention Layers
- 8 End-to-End PyTorch Model
- 9 What You Should Remember

I. Recall: Gradient Descent

Audience question

What does it mean mathematically to “Performing a Cognitive Task”?

Audience question

What does gradient descent do during training?

Mathematical view

Many supervised learning problems can be written as:

$$\hat{y} = f_{\theta}(x)$$

where:

- x is the input,
- $\hat{y}$ is the model prediction,
- θ are the trainable parameters.

The objective is to reduce the difference between $\hat{y}$ and the target y using a loss function:

$$\mathcal{L}(y, \hat{y}).$$

Gradient Descent Reminder

Parameter update

For a parameter w , gradient descent applies:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

where η is the learning rate.

For a linear regression:

$$\hat{y} = wx + b$$

we update both parameters:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}.$$

Gradient Descent in PyTorch

```
import torch
import torch.nn as nn

model = nn.Linear(1, 1)
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

x = torch.randn(100, 1)
y = 2 * x + 1

for epoch in range(100):
    y_pred = model(x) # forward pass: compute predictions
    loss = criterion(y_pred, y) # compute the training error

    optimizer.zero_grad() # reset old gradients
    loss.backward() # compute new gradients
    optimizer.step() # update model parameters
```

II. Training Vocabulary

Audience question

What is the difference between an epoch and a batch?

Epoch and Batch Size

Epoch

One epoch means that the model has seen the entire training dataset once.

Batch size

The batch size is the number of training examples used to compute one gradient update.

Example:

- dataset size: 1000 examples,
- batch size: 100,
- one epoch: 10 gradient updates.

Key message

Training usually updates the model several times per epoch, using mini-batches instead of the full dataset at once.

Mini-Batch Training Loop

```
for epoch in range(num_epochs):  
    for x_batch, y_batch in dataloader:  
        pred = model(x_batch)  
        loss = criterion(pred, y_batch)  
  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()
```

Practical meaning

Each batch produces one loss, one backward pass, and one optimizer step.

III. Linear Regression as a Neuron

Section objective

Understand why a simple linear regression can be interpreted as a neural model.

What is a Neuron?

Artificial neuron

An artificial neuron receives input values, combines them using trainable weights, adds a bias, and produces an output.

For inputs $x_1, \dots, x_d$:

$$z = \sum_{i=1}^d w_i x_i + b$$

The neuron output is often:

$$h = \phi(z)$$

where ϕ is an activation function.

Key idea

A neuron is a small parametric function with weights and a bias.

Audience question

How many neurons are there in a standard linear regression?

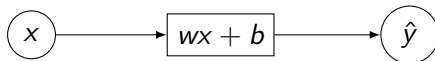
Linear Regression as One Neuron

A linear regression with one input is:

$$\hat{y} = wx + b.$$

This is exactly one neuron without activation:

$$z = wx + b.$$



If x has several inputs (this is called a *feature map*), we define the linear regression with:

$$y = \sum_{i=1}^{d_{in}} w_i x_i + b$$

Answer

A standard scalar linear regression can be viewed as one neuron.

From One Neuron to One Layer

Most machine learning tasks require multiple outputs and therefore multiple neurons working in parallel.

By stacking multiple neurons together, we obtain a linear layer:

$$Y = XW^{\top} + b$$

where

$$X_{(N, d_{in})}, \quad W_{(d_{out}, d_{in})}, \quad Y_{(N, d_{out})}$$

Why the transpose?

PyTorch stores the weight matrix as $W \in \mathbb{R}^{d_{out} \times d_{in}}$, where each row corresponds to one output neuron; therefore the computation uses $Y = XW^{\top} + b$. The mathematical notation is $Y = XW + b$ with $W \in \mathbb{R}^{d_{in} \times d_{out}}$

Example: PyTorch Linear Layer

```
import torch
import torch.nn as nn

layer = nn.Linear(64, 256)
x = torch.randn(64)
y = layer(x)

print(layer.weight.shape) # ?
print(layer.bias.shape) # ?
print(y.shape) # ?
```

Question

What are the dimensions of the weights, bias, and output?

Example: PyTorch Linear Layer – Answer

```
import torch
import torch.nn as nn

layer = nn.Linear(64, 256)
x = torch.randn(64)
y = layer(x)

print(layer.weight.shape) # torch.Size([256, 64])
print(layer.bias.shape) # torch.Size([256])
print(y.shape) # torch.Size([256])
```

$$W \in \mathbb{R}^{256 \times 64}, \quad b \in \mathbb{R}^{256}, \quad y \in \mathbb{R}^{256}.$$

Key message

`nn.Linear(64, 256)` learns 256 neurons, each connected to the 64 input features.

Training a Linear Layer

```
model = nn.Linear(64, 10)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

x = torch.randn(64)
target = torch.randn(10)

for epoch in range(50):
    pred = model(x)
    loss = criterion(pred, target)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

IV. Activation Functions

Audience question

What happens if we stack 2 linear layers?

Consider a neural network with two linear layers:

$$h = W_1x + b_1$$

$$y = W_2h + b_2$$

Substituting the first equation into the second:

$$y = W_2(W_1x + b_1) + b_2$$

$$y = (W_2W_1)x + (W_2b_1 + b_2)$$

Example : ReLU Activation Function

Definition

$$\text{ReLU}(x) = \max(0, x)$$

- negative values become 0,
- positive values are kept,
- the model becomes non-linear.

Key message

Activation functions add non-linearity to the model. Without them, a deep linear network remains equivalent to one linear layer. Pytorch ReLU

Linear Layer + Activation

```
model = nn.Sequential(  
    nn.Linear(64, 256),  
    nn.ReLU(),  
    nn.Linear(256, 10)  
)
```

Interpretation

- `Linear(64, 256)` learns 256 linear combinations.
- `ReLU()` adds non-linearity.
- `Linear(256, 10)` produces the final output.

V. What is an Image Mathematically?

A color image is not just one number. It is usually represented as a 3D tensor:

$$X \in \mathbb{R}^{C \times H \times W}$$

where:

- C is the number of channels, for example RGB gives $C = 3$,
- H is the image height,
- W is the image width.

Example:

$$X \in \mathbb{R}^{3 \times 32 \times 32}$$

means a color image with 3 channels and 32×32 pixels.

Audience question

Why are linear layers not ideal for images?

Image Classification Example

Task

Given an image, predict a class:

$$f_{\theta}(x) \rightarrow \{\text{cat}, \text{dog}\}$$

Images have spatial structure:

- local patterns,
- edges and textures,
- shapes and objects.



Convolution Layers

A fully connected layer sees the image as a flat vector.

A convolutional layer preserves spatial structure by applying local filters.

Key idea

A convolution is a local linear operation with shared weights.

Useful visualization resource:

<https://animatedai.github.io/>

Convolution Equation

For an input feature map:

$$X \in \mathbb{R}^{C_{in} \times H \times W}$$

and a kernel:

$$K \in \mathbb{R}^{C_{out} \times C_{in} \times k_H \times k_W}$$

the output is:

$$Y_{C_{out},u,v} = b_{C_{out}} + \sum_{c=1}^{C_{in}} \sum_{r=1}^{k_H} \sum_{s=1}^{k_W} K_{C_{out},c,r,s} \times X_{c,u+r,v+s}.$$

Interpretation

Each output value is computed from a local patch of the input image.

Kernel Size and Padding

Kernel size

The `kernel_size` defines the spatial size of the local window. For example, `kernel_size=3` means a 3×3 filter.

Padding

Padding adds artificial values around the image borders. It controls whether the output spatial size shrinks or stays similar.

For stride 1:

$$H_{out} = H + 2p - k + 1$$

- no padding: output becomes smaller,
- padding $p = 1$ with $k = 3$: output keeps the same size.

Example: PyTorch Convolution Layer

```
import torch
import torch.nn as nn

conv = nn.Conv2d(
    in_channels=3,
    out_channels=16,
    kernel_size=3,
    padding=1
)

x = torch.randn(3, 32, 32)
y = conv(x)

print(conv.weight.shape) # ?
print(conv.bias.shape) # ?
print(y.shape) # ?
```

Question

What are the dimensions of the filters, bias, and output feature map?

Example: PyTorch Convolution Layer – Answer

```
import torch
import torch.nn as nn

conv = nn.Conv2d(
    in_channels=3,
    out_channels=16,
    kernel_size=3,
    padding=1
)

x = torch.randn(3, 32, 32)
y = conv(x)

print(conv.weight.shape) # torch.Size([16, 3, 3, 3])
print(conv.bias.shape) # torch.Size([16])
print(y.shape) # torch.Size([16, 32, 32])
```

Key message

The convolution learns 16 filters. Each filter looks at the 3 input channels with a 3×3 window.

Training a Convolution Layer

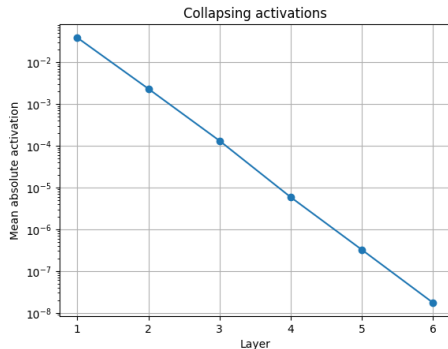
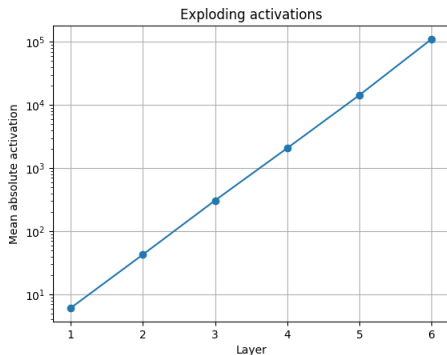
```
model = nn.Sequential(  
    nn.Conv2d(3, 16, kernel_size=3, padding=1),  
    nn.ReLU(),  
    nn.Conv2d(16, 2, kernel_size=3, padding=1)  
)  
  
criterion = nn.MSELoss()  
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)  
  
x = torch.randn(3, 32, 32)  
target = torch.randn(2, 32, 32)  
  
pred = model(x)  
loss = criterion(pred, target)  
  
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

VI. Normalization Layers

Audience question

During training, the distribution of features may change from one iteration to another. Why can this make learning more difficult?

Exploding and Collapsing Values



Observation

Without normalization, activations can grow very quickly or collapse towards zero across layers.

BatchNorm and LayerNorm

Batch Normalization

Normalizes using statistics computed across a mini-batch.

Common in convolutional neural networks.

Layer Normalization

Normalizes features within one example.

Common in attention-based models and transformers.

Practical intuition

Normalization does not replace learning. It helps the learning process by controlling the scale of internal representations.

BatchNorm Example

Normalization layers help by keeping values in a more stable range.

General idea

Normalize activations using a mean and a variance:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

then learn a scale and shift:

$$y = \gamma \hat{x} + \beta.$$

Key message

Normalization can make optimization more stable and often improves convergence.

BatchNorm vs Linear Layer

A linear layer mixes the input features:

$$y = Wx + b$$

For example:

$$y_1 = w_{11}x_1 + w_{12}x_2 + \dots + w_{1d}x_d$$

Each output can depend on all input features.

BatchNorm does not mix features:

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

Each feature is normalized and transformed independently.

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \rightarrow \begin{pmatrix} \gamma_1 \hat{x}_1 + \beta_1 \\ \gamma_2 \hat{x}_2 + \beta_2 \\ \gamma_3 \hat{x}_3 + \beta_3 \end{pmatrix}$$

BatchNorm vs LayerNorm : 3 samples for 4 features

BatchNorm

Normalizes each feature across the batch.

$$\begin{bmatrix} \textcolor{red}{x}_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ \textcolor{red}{x}_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \\ \textcolor{red}{x}_{3,1} & x_{3,2} & x_{3,3} & x_{3,4} \\ \textcolor{red}{x}_{4,1} & x_{4,2} & x_{4,3} & x_{4,4} \end{bmatrix}$$

Normalize one column at a time.

LayerNorm

Normalizes all features of one sample.

$$\begin{bmatrix} \textcolor{blue}{x}_{1,1} & \textcolor{blue}{x}_{1,2} & \textcolor{blue}{x}_{1,3} & \textcolor{blue}{x}_{1,4} \\ x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \\ x_{3,1} & x_{3,2} & x_{3,3} & x_{3,4} \\ x_{4,1} & x_{4,2} & x_{4,3} & x_{4,4} \end{bmatrix}$$

Normalize one row at a time.

Key idea

BatchNorm normalizes **columns** (features across samples).

LayerNorm normalizes **rows** (features within a sample).

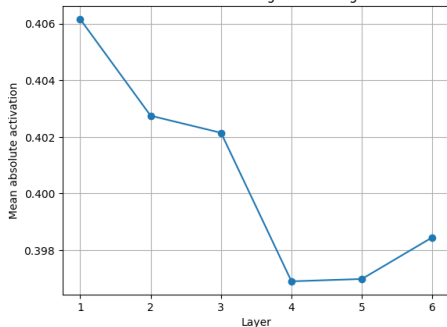
Normalization in PyTorch

```
# Example with linear layers
model = nn.Sequential(
    nn.Linear(64, 256),
    nn.LayerNorm(256),
    nn.ReLU(),
    nn.Linear(256, 10)
)

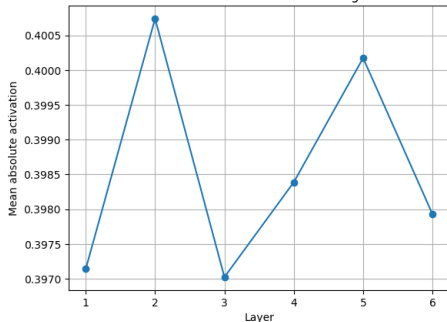
# Example with convolution layers
cnn = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1),
    nn.BatchNorm2d(16),
    nn.ReLU(),
    nn.Conv2d(16, 32, kernel_size=3, padding=1)
)
```

Effect of Batch Normalization

With BatchNorm: large initial weights



With BatchNorm: small initial weights



Observation

With BatchNorm, activation scales remain much more stable across layers.

VII. Attention Layers

Audience question

How can a model decide which parts of an input are the most important?

Motivation: A Sentence Example

Consider the sentence:

“The animal did not cross the street because it was too tired.”

To understand the word *it*, the model needs context.

Attention allows one token to look at other tokens and assign them different importance scores.

Key idea

Attention is a mechanism for weighted information selection. It is the layer that enabled today's LLMs.

Original paper: *Attention Is All You Need* (Vaswani et al., 2017).

Query, Key, Value

In attention, each token is projected into three vectors:

- Query Q : what this token is looking for,
- Key K : what this token contains,
- Value V : the information to transmit.

Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

where d_k is the dimension of the key and query vectors. It is used to scale the dot product and avoid overly large attention scores.

Interpretation

The softmax produces attention weights. These weights decide how much information is taken from each value vector.

Example: PyTorch Multi-Head Attention

```
embed_dim = 64
num_heads = 8

attention = nn.MultiheadAttention(
    embed_dim=embed_dim,
    num_heads=num_heads,
    batch_first=True
)

x = torch.randn(32, 10, 64) # batch, sequence length, embedding dim

output, weights = attention(x, x, x)

print(output.shape) # torch.Size([32, 10, 64])
print(weights.shape) # torch.Size([32, 10, 10])
```

Interpretation

Each token attends to all tokens in the sequence. Here, the model processes a batch of 32 sequences, each containing 10 tokens represented by vectors of dimension 64.

VIII. End-to-End PyTorch Model

Audience question

Can we now identify the main steps of a complete training loop?

A Complete Training Loop

```
model = nn.Sequential(  
    nn.Linear(64, 256),  
    nn.LayerNorm(256),  
    nn.ReLU(),  
    nn.Linear(256, 10)  
)  
  
criterion = nn.MSELoss()  
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)  
  
for epoch in range(num_epochs):  
    for x_batch, y_batch in dataloader:  
        pred = model(x_batch)  
        loss = criterion(pred, y_batch)  
  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()
```

IX. What You Should Remember

- 1 Linear regression can be viewed as one neuron.
- 2 A linear layer is a set of neurons applied in parallel.
- 3 Activation functions add non-linearity to the model.
- 4 Epoch and batch size describe how training data is used.
- 5 Convolution layers exploit spatial structure with local filters.
- 6 Normalization layers help stabilize training.
- 7 Attention layers select useful information using learned weights.
- 8 PyTorch training requires: forward, loss, zero_grad, backward, and step.

References

- PyTorch documentation: `torch.nn.Linear`.
<https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html>
- PyTorch documentation: `torch.nn.Conv2d`.
<https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>
- PyTorch documentation: `torch.optim`.
<https://docs.pytorch.org/docs/stable/optim.html>
- PyTorch documentation: `torch.nn.MultiheadAttention`.
<https://docs.pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html>
- Visualization resource for convolution and neural networks.
<https://animatedai.github.io/>
- A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.
<https://arxiv.org/abs/1706.03762>
- S. Ioffe and C. Szegedy. *Batch Normalization*. ICML, 2015.
- J. L. Ba, J. R. Kiros, G. E. Hinton. *Layer Normalization*. 2016.